Class: SE **Subject: COA LAB** **Sem-III**

Experiment No 9

Course Outcome: CO5 **Blooms Level:L3**

Aim: Implement Booth's Multiplication Algorithm

Abstract: Develop an 8086 assembly language program to perform signed multiplication using Booth's Multiplication Algorithm. The program takes two signed 8-bit numbers in two's complement form as inputs and multiplies them using Booth's method, which efficiently handles both positive and negative numbers by combining arithmetic and logical operations. The implementation utilizes shifting, addition, and subtraction operations based on the multiplier bits and uses appropriate register-based addressing and looping techniques. The final product is stored in a 16-bit register pair, accurately representing the signed result of the multiplication.

Sample Input and Output:
Case 1: Q=7, M=5
Case 2: Q=2, M=8

Theory:
Booth's Algorithm is a multiplication algorithm that provides an efficient way to multiply binary numbers, especially when dealing with signed (positive and negative) numbers. It optimizes the process by reducing the number of additions and subtractions required, making it faster than traditional multiplication methods. This algorithm is commonly used in computer architecture for signed multiplication.

Booth's algorithm efficiently handles both positive and negative numbers in 2's complement representation, which is crucial for signed multiplication in computer systems. By cleverly recoding the multiplier and using a combination of addition and subtraction, it significantly reduces the number of arithmetic operations compared to straightforward multiplication methods. The algorithm's structure makes it well-suited for implementation in hardware, contributing to faster multiplication in processors.

Booth's algorithm accepts positive and negative numbers. Negative numbers must be represented in 2's complement form. The range of an n-bit two's complement number is from $-2^{(n-1)}$ to $2^{(n-1)} - 1$. For example, 8-bit two's complement number can represent values from -128 to 127.

In 8086 assembly, this algorithm is implemented using registers for the accumulator, multiplier, multiplicand, and a counter to track shifts. Q and M are stores in data segment, as a multiplicand and a multiplier, respectively. The carry flag bit of the status register is used to store Q_{-1} bit of the algorithm. AH and AL registers store A and Q values of the algorithm.

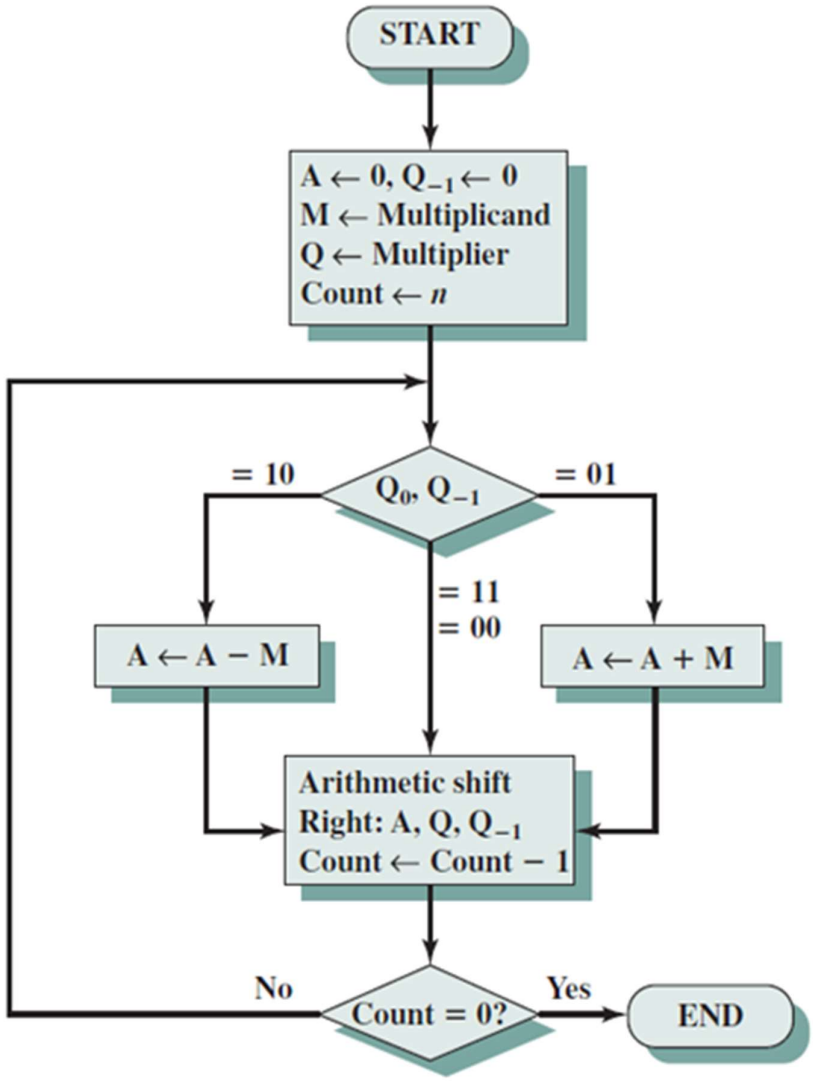


Figure 1: Booth's Algorithm

Program: Assembly program to implement multiplication of two 8 bit numbers using Booths algorithm.

DATA SEGMENT

```
Q  DB 8H
M DB 5H
RESULT DW  ?
    DB 100 DUP(00)
DATA ENDS

CODE SEGMENT
ASSUME CS: CODE, DS: DATA

START:  MOV AX, DATA
        MOV DS, AX

        MOV AX, 0000H
        MOV BX, 0000H
        MOV AL, Q
        MOV BL, M
        MOV CL, 8
        CLC
BACK: MOV DX, 00H
        MOV DX, AX
        RCL DX, 01
        AND DX, 03H

        CMP DX, 01H
        JE Q01
        CMP DX, 02H
        JE Q10
NEXT: SAR AX, 01
        LOOP BACK
        JMP EXIT

Q10:    SUB AH, BL
        JMP NEXT
Q01:    ADD AH, BL
        JMP NEXT

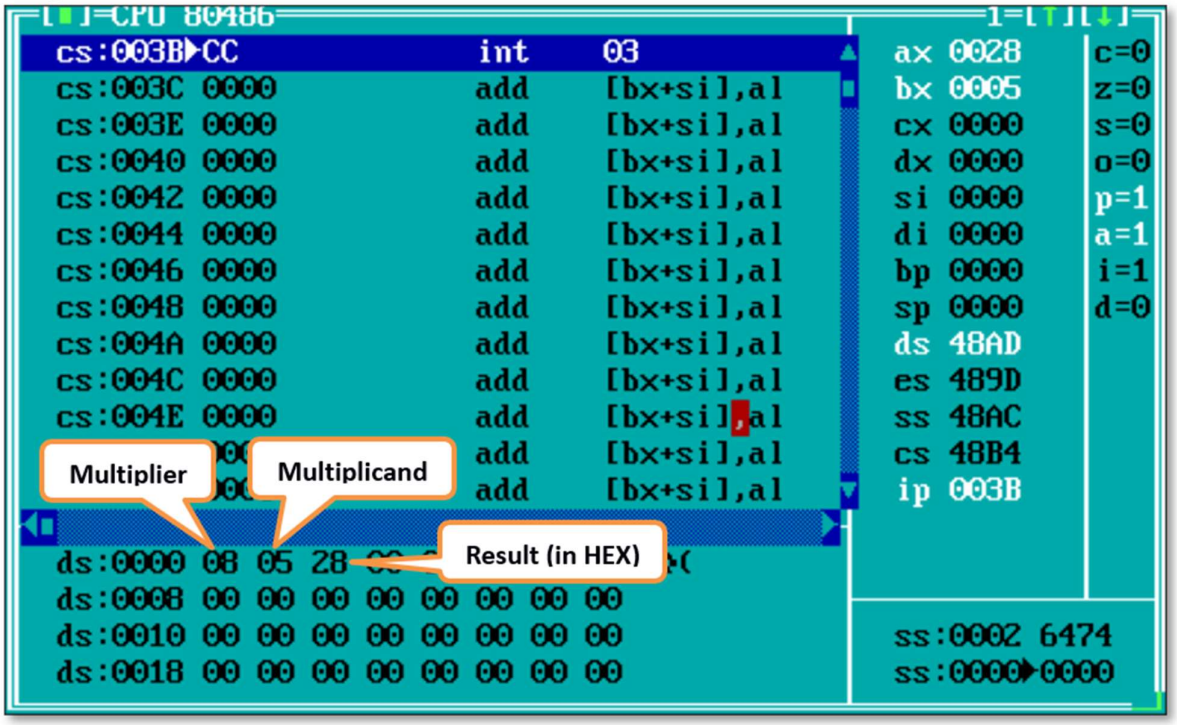
EXIT:  MOV RESULT, AX

        INT 3H
CODE ENDS
END START
```

Output:

Class: SE **Subject: COA LAB** **Sem-III**

Output Window:



Conclusion: The experiment on "Implementation of Booth's Multiplication Algorithm" was successfully performed using 8086 assembly language. Booth's algorithm enabled efficient multiplication of signed binary numbers by applying a sequence of additions, subtractions, and arithmetic shifts based on bit patterns of the multiplier. The program correctly interpreted two's complement representations to handle positive and negative numbers, showing the ability of Booth's method to minimize the number of arithmetic operations.

Class: SE **Subject: COA LAB** **Sem-III**

Exercise 1: Develop an assembly language program to implement the Restoring-based division operation.

Abstract: Develop an 8086 assembly language program to perform unsigned binary division using the Restoring Division Algorithm. The program takes two 8-bit unsigned binary numbers as the dividend and divisor, and performs step-by-step division using shifting and subtraction operations as per the restoring technique. The quotient and remainder are obtained through iterative comparison and condition-based restoration of the partial remainder.